



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

PROJECT 2 REPORT

ECTE441 – Intelligent Control

Student Name	Student Number
Yasan Gunathilaka	7549246

06th June 2025

I. Table of Contents

I.	TABLE OF CONTENTS	II
II.	TABLE OF FIGURES	III
1.	INTRODUCTION	1
2.	TASK 1 – FUZZY CONTROL SIMULATION	1
3.	TASK 2 – FUZZY ANFIS CONTROL	4
4.	CONCLUSION	8
5.	APPENDIX.....	9
A.	TASK 1: FUZZY CONTROL SIMULATION	9
B.	TASK 2: FUZZY ANFIS CONTROL.....	13

II. Table of Figures

FIGURE 2.1 - SIMULINK MODEL FOR THE ROBOT USING FUZZY CONTROLLER	1
FIGURE 2.2 - FUZZY CONTROLLER DESIGN AND ITS MEMBERSHIP FUNCTIONS	2
FIGURE 2.3 - FUZZY CONTROLLER RULES.....	3
FIGURE 2.4 - TRAJECTORY OF THE ROBOT USING A FUZZY CONTROLLER	3
FIGURE 3.1 - SIMULINK MODEL FOR THE ROBOT USING ANFIS CONTROLLER	4
FIGURE 3.2 – TRAINING THE WHEELS FOR ANFIS CONTROLLER: I) LEFT WHEEL – 400 EPOCHS, 0.159 ERROR (LEFT), II) RIGHT WHEEL – 400 EPOCHS, 0.414 ERROR (RIGHT)	5
FIGURE 3.3 - ANFIS CONTROLLER MEMBERSHIP FUNCTIONS AND 164 RULES: I) LEFT WHEEL (VL) II) RIGHT WHEEL (VR).....	6
FIGURE 3.4 - TRAJECTORY OF THE ROBOT USING ANFIS CONTROLLER.....	7
FIGURE 5.1 - NINE ADDITIONAL RUNS FROM DIFFERENT LOCATIONS AT DIFFERENT ANGLES	10
FIGURE 5.2 - INPUT AND OUTPUT SIGNALS OF THE ROBOT WITH A FUZZY CONTROLLER.....	11
FIGURE 5.3 – FUZZY RULE SURFACE VIEW OF LEFT WHEEL: I) VL PLOTTED AGAINST THETA (X-AXIS) AND DIS (Y-AXIS), II) VL PLOTTED AGAINST OB_LOC (X-AXIS) AND ULTRASONIC (Y-AXIS) - (RIGHT)	12
FIGURE 5.4 – FUZZY RULE SURFACE VIEW OF RIGHT WHEEL: I) VR PLOTTED AGAINST THETA (X-AXIS) AND DIS (Y-AXIS), II) VR PLOTTED AGAINST OB_LOC (X-AXIS) AND ULTRASONIC (Y-AXIS) - (RIGHT)	12
FIGURE 5.5 - INPUT AND OUTPUT SIGNALS OF THE ROBOT WITH ANFIS CONTROLLER.....	13
FIGURE 5.6 - ANFIS RULE SURFACE VIEW OF LEFT WHEEL: I) VL PLOTTED AGAINST THETA (X-AXIS) AND DIS (Y-AXIS), II) VL PLOTTED AGAINST OB_LOC (X-AXIS) AND ULTRASONIC (Y-AXIS) - (RIGHT)	14
FIGURE 5.7 - ANFIS RULE SURFACE VIEW OF RIGHT WHEEL: I) VR PLOTTED AGAINST THETA (X-AXIS) AND DIS (Y-AXIS), II) VR PLOTTED AGAINST OB_LOC (X-AXIS) AND ULTRASONIC (Y-AXIS) - (RIGHT)	14

1. Introduction

This project aims to control the left and right wheels of a mobile robot in a two-dimensional space by initially developing a general fuzzy controller and then using its data to create an Adaptive Neural Fuzzy Inference System (ANFIS) controller. The function of the robot is to reach its target while avoiding the fourteen obstacles (mines) in its way without utilising any coordinate positions.

The MATLAB Simulink environment models the robot as a multi-input multi-output (MIMO) system – this is represented by the ‘RoboBlock.slx’ file, which incorporates the following outputs:

- dis – estimated line-of-sight distance between the robot and the target
- odometer – total accumulated distance travelled by the robot
- ultrasonic – measures the proximity of the obstacles directly in front of the robot
- theta – angle between the direction the robot is facing and the target
- ob_loc (activeSensor) – determines the location of the obstacle relative to the robot

2. Task 1 – Fuzzy Control Simulation

The fuzzy controller is designed by utilising four out of the five signals; dis, ultrasonic, theta and activeSensor; the odometer is not connected as it is deemed not as useful to reach the target. The starting coordinates are set to $X = 3.4$ and $Y = 5.8$, and the model is initially tuned in a five-obstacle course. The Simulink model is displayed in *Figure 2.1*.

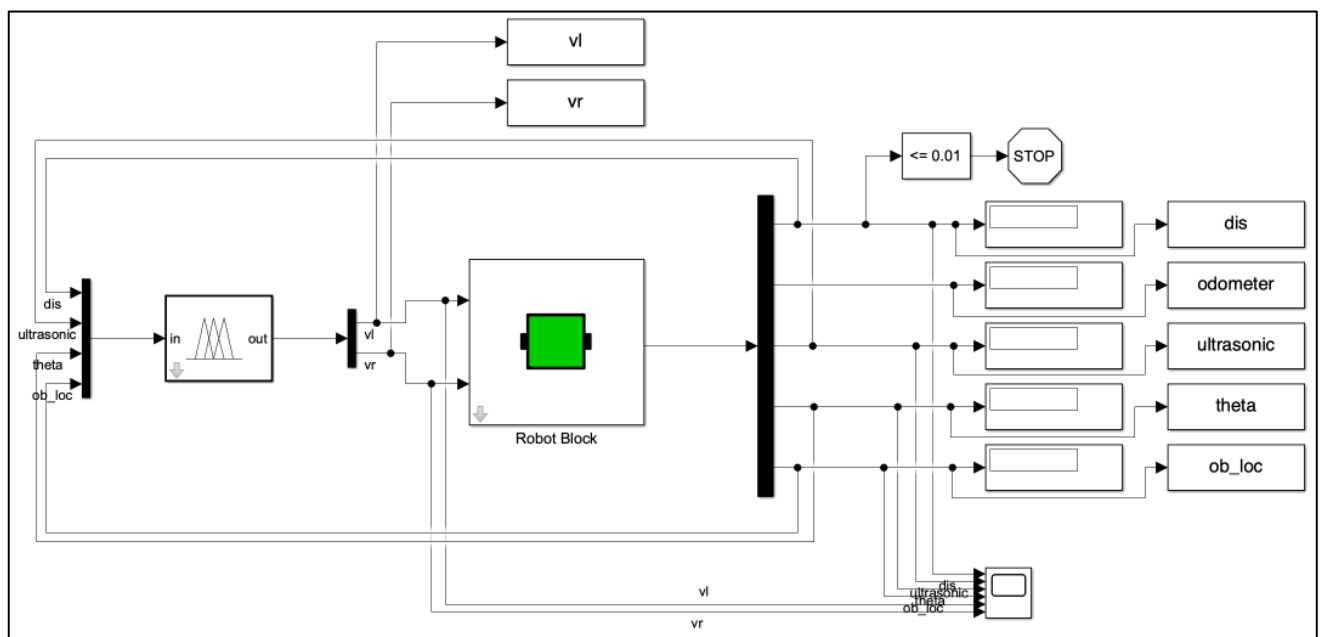


Figure 2.1 - Simulink model for the robot using fuzzy controller

A ‘scope’ block is used to visualise the input and outputs over time in a graph; a ‘stop’ block is implemented such that the simulation stops when the robot reaches its target (i.e. when dis goes below 0.01 m). ‘To workspace’ blocks are used to store the simulation data in the MATLAB workspace so they can be utilised for Task 2.

The design of the fuzzy controller is displayed in *Figure 2.2*. It uses a centroid defuzzification with min for AND, max for OR, and max for aggregation, to ensure smooth and balanced outputs while also prioritising responsive decision making. For simplicity, triangular membership functions were used.

except for the theta input, where ‘target left’ and ‘target right’ were given a wider region using the trapezoidal function, as a narrow region was assigned for the ‘target centre’ to prioritise the change of direction towards the target.

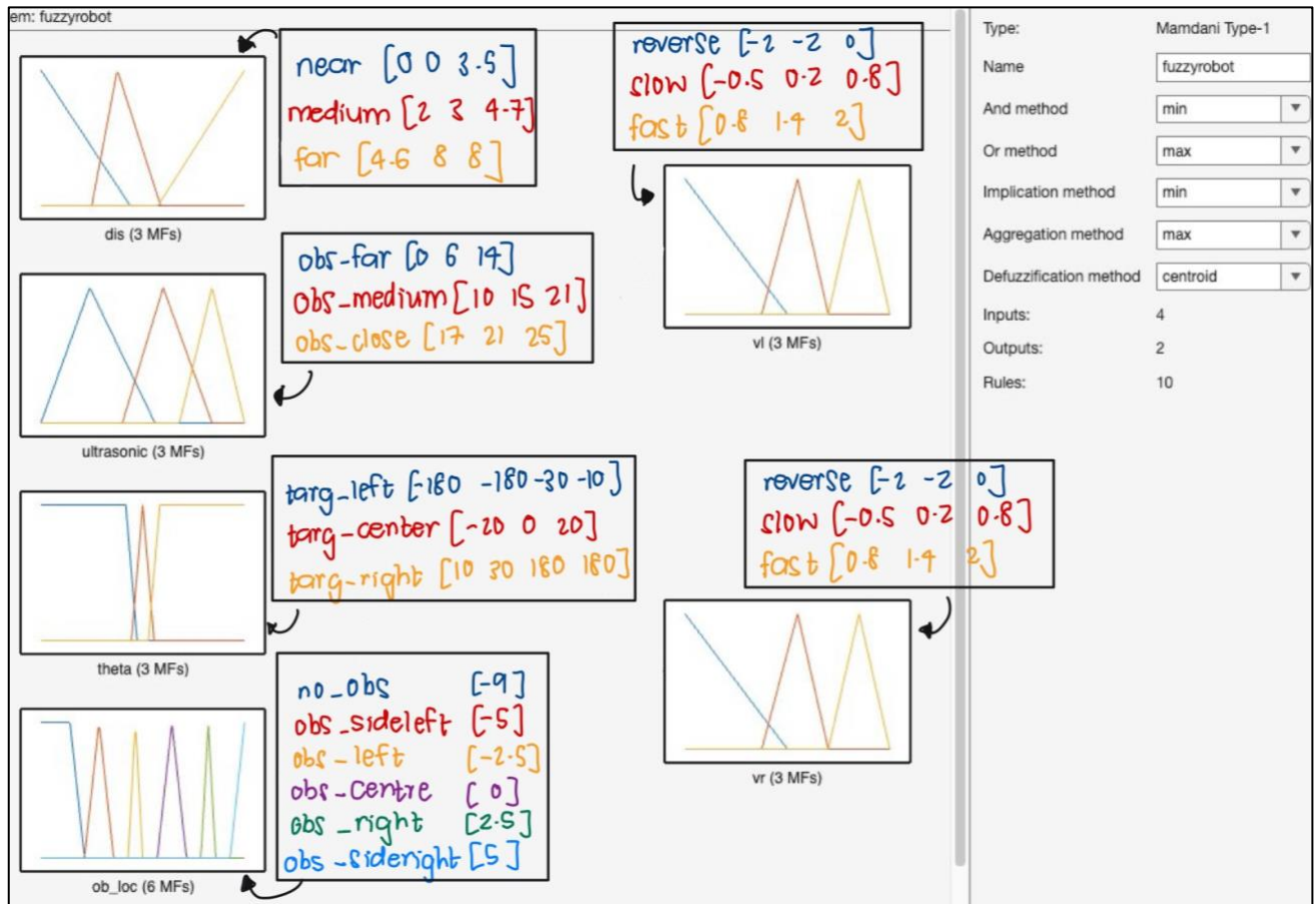


Figure 2.2 - Fuzzy controller design and its membership functions

Each input and output was given three membership functions each, except for the activeSensor, which was given six narrow functions to match the fixed discrete values returned from the sensor. The ‘reverse’ regions of the outputs were extended to represent all the negative values within the range of -2 to 2, however, the ‘reverse’ function led to many complexities and unexpected robot behaviour. Therefore, its intensity was reduced by limiting its peak at -2. Figure 2.3 highlights the fuzzy rules implemented.

	Rule	Weig...	Name
1	If dis is far and ultrasonic is obs_close and ob_loc is obs_centre then vl is slow, vr is reverse	1	rule1
2	If ultrasonic is obs_close and ob_loc is obs_left then vl is fast, vr is slow	1	rule2
3	If ultrasonic is obs_close and ob_loc is obs_right then vl is slow, vr is fast	1	rule3
4	If ultrasonic is obs_close and ob_loc is obs_sideleft then vl is fast, vr is slow	1	rule4
5	If ultrasonic is obs_close and ob_loc is obs_sideright then vl is slow, vr is fast	1	rule5
6	If ultrasonic is not obs_close and theta is targ_left then vl is slow, vr is fast	1	rule6
7	If ultrasonic is not obs_close and theta is targ_right then vl is fast, vr is reverse	1	rule7
8	If dis is near and ultrasonic is obs_close and ob_loc is obs_centre then vl is reverse, vr is fast	1	rule8
9	If ultrasonic is not obs_close and theta is targ_center then vl is fast, vr is fast	1	rule9
10	If dis is medium and ultrasonic is obs_close and ob_loc is obs_centre then vl is reverse, vr is fast	1	rule10

Figure 2.3 - Fuzzy controller rules

The robot performed beyond expectations, plotting the path displayed in Figure 2.4.

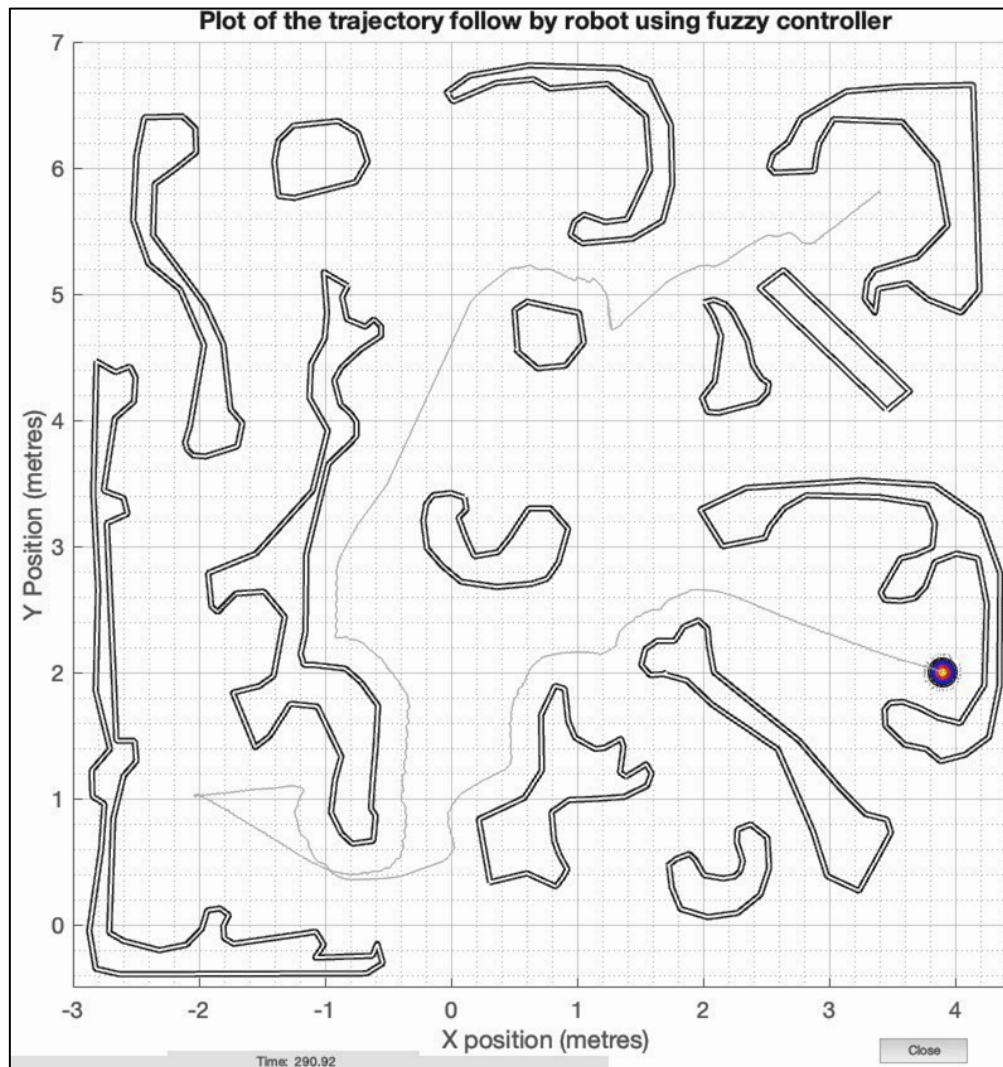


Figure 2.4 - Trajectory of the robot using a fuzzy controller

The simulation was executed with a simulation override step of 0.04 for faster results. Nonetheless, the robot consistently reached the target at both the default speed and speeds slightly higher than 0.04 from any location of the obstacle course. The trajectories of the 9 additional locations, each with a different

starting angle were used as data for Task 2, the plots are displayed in *Figure 5.1, Appendix A*. Locations were picked to be very different to one another, this would influence a lot of variation in the data, aiding better training of the ANFIS controller. At significantly higher speeds, the robot exhibited unnatural behaviour and a tendency to spiral. The goal was reached in 291 seconds with an odometer reading of **20.36 metres**.

Figure 5.2 in Appendix A displays the scope showing all input and output signals (excluding the odometer) throughout the robot's journey from start to destination. It was observed that due to the narrow triangular membership function assigned to 'target centre' in theta, the fuzzy controller reacted very aggressively with sharp turns and oscillatory movement in certain areas. This is highlighted by the noisy signal observed in the graph. *Figure 5.3 and Figure 5.4 in Appendix A* show the rule surface views for the left and right wheels, illustrating how vl or vr responds when two out of the four inputs (theta, dis, ob_loc and ultrasonic) are varied while the others are left constant. The simplicity of the control system is highlighted, as both wheels follow nearly mirrored surface patterns. Upon completion, it was determined that a zero-order Sugeno (TSK) controller may have been easier to fine-tune over the chosen first-order Mamdani; it would also be easier to apply the Sugeno controller when designing the ANFIS controller in Task 2. Additionally, the system could have been further simplified by having a single output instead of two separate outputs for each wheel. This single output could have later been branched off into two separate outputs for each wheel.

3. Task 2 – Fuzzy ANFIS Control

The fuzzy controller from Task 1 is used as an aid to train and develop an ANFIS controller in Task 2. The Simulink model is adjusted such that each wheel is controlled by its own ANFIS controller, this can be seen in *Figure 3.1*.

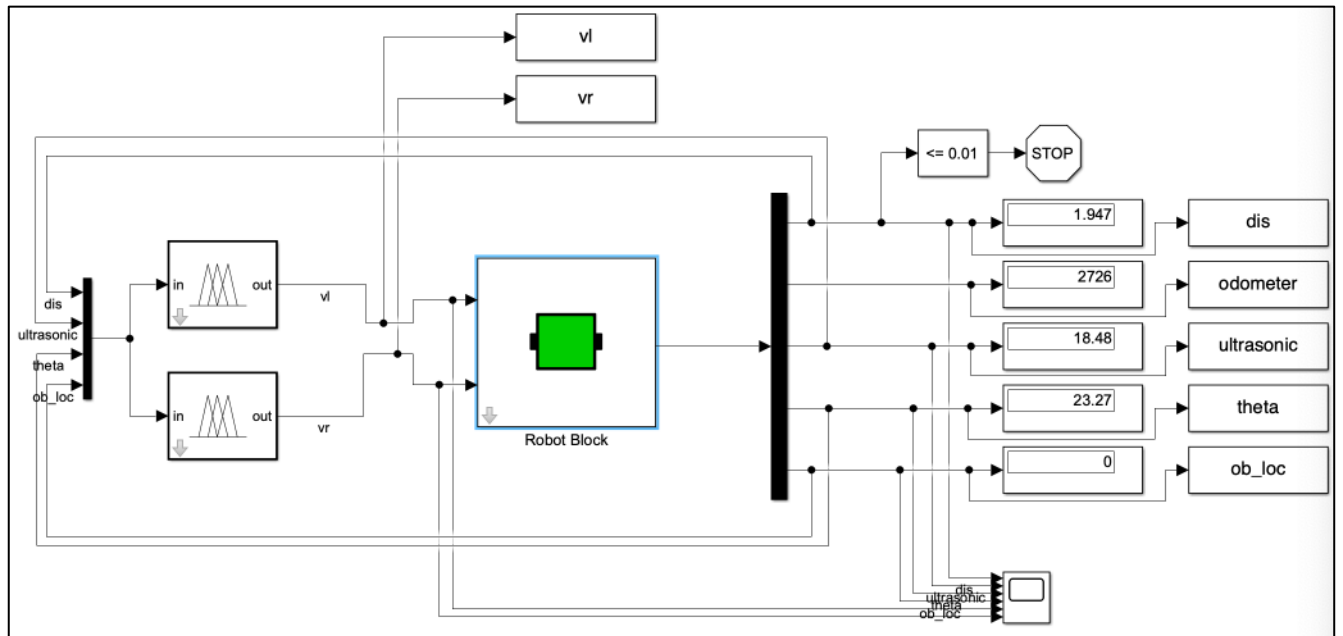


Figure 3.1 - Simulink model for the robot using ANFIS controller

A conversion of the Task 1 controller into a zero-order (constant) Sugeno is used as the base FIS. The inputs (dis, ultrasonic, theta, and ob_loc) and outputs (vl and vr) saved from each run in Task 1 are extracted into an array and fed to 'anfisedit' to optimise the new controller. While the odometer data

could have further improved the system, this was negated as the fuzzy controller operated with minimal loops and therefore did not accumulate high odometer values for most locations.

Each wheel was initially trained over 100 epochs. However, as the training error graph did not show signs of convergence, this value was gradually increased to 400 epochs. While the error continued to decrease, the rate of improvement was extremely slow, making further training inefficient and computationally demanding. As a result, training was halted at **400 epochs** for each wheel. The final training error was **0.159** for the left wheel and a significantly higher **0.414** for the right wheel. The results are displayed in *Figure 3.2*.

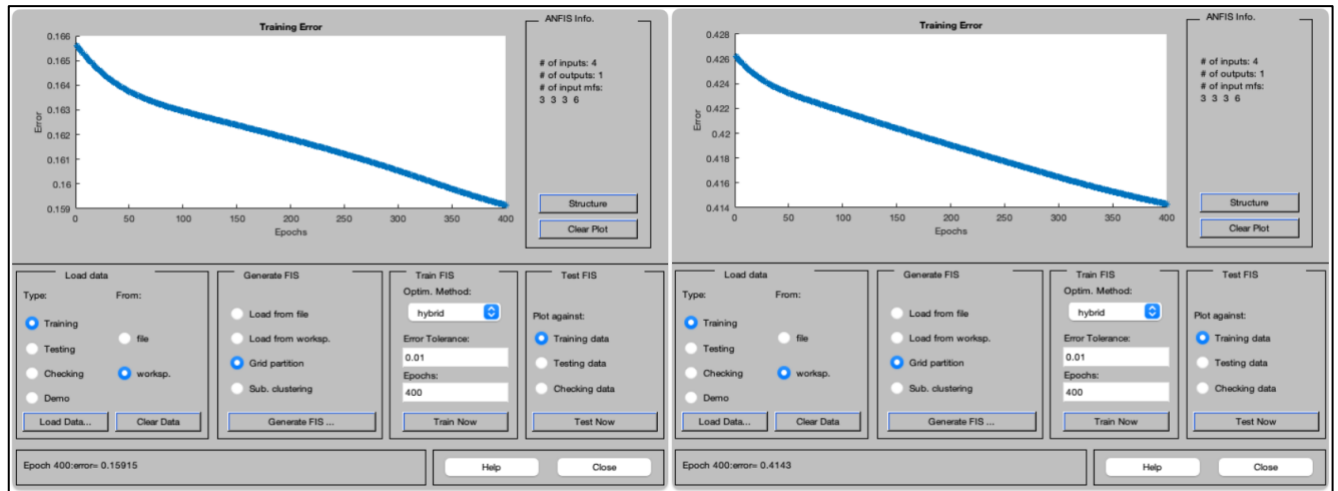


Figure 3.2 – Training the wheels for ANFIS controller: i) Left wheel – 400 epochs, 0.159 error (left), ii) Right wheel – 400 epochs, 0.414 error (right)

The membership functions in the ANFIS controller were initially very similar to the fuzzy. Upon training the model, the membership functions were noticeably smoother and evenly distributed with overlap in comparison. The number of membership functions remained the same, however, Gaussian functions were used instead of the sharp triangular functions originally selected by the student. A significant change was observed in the range of the ‘distance to target’: the trained ANFIS controller operated over a much larger range than the initially defined [0, 8]. This likely reflects the large, obstacle-free workspace available to the robot, even though the actual obstacles and target are located within a smaller area.

The defuzzification method was changed from ‘centroid’ to ‘wtaver’ (weighted average), with the logical AND operator set to ‘prod’ and the OR operator to ‘probor’. While it’s debatable whether these changes directly improved performance, the resulting controller was noticeably more stable.

Unlike the 10 intuitive rules that were designed by the student in the original fuzzy, each ANFIS controller generated over 50 different rules. Although only a few were actively fired during operation, the presence of these additional (and sometimes redundant) rules ensured that at least one rule would fire in any situation (e.g. when the robot is very far away from the target in the workspace). The membership functions and corresponding rules for each wheel are shown in *Figure 3.3*.



Figure 3.3 - ANFIS controller membership functions and 164 rules: i) left wheel (vl) ii) right wheel (vr)

The simulation was run and robot's trajectory was plotted. With the ANFIS controller, the robot moved more confidently and precisely, avoiding the sharp, unstable turns observed with the fuzzy controller. However, it followed a longer path overall, resulting in a final odometer reading of **31.86 metres**.

In contrast to the fuzzy controller, this system demonstrated stronger obstacle avoidance - slowing down when an obstacle was detected and proceeding with caution. The route taken in the final portion of the simulation closely resembled that of the fuzzy controller. This is likely because, even though there were differences in the training data locations, the final navigation segment (between the initial and final target) remained consistent across all simulations. A plot of the trajectory followed by the robot is displayed in *Figure 3.4*.

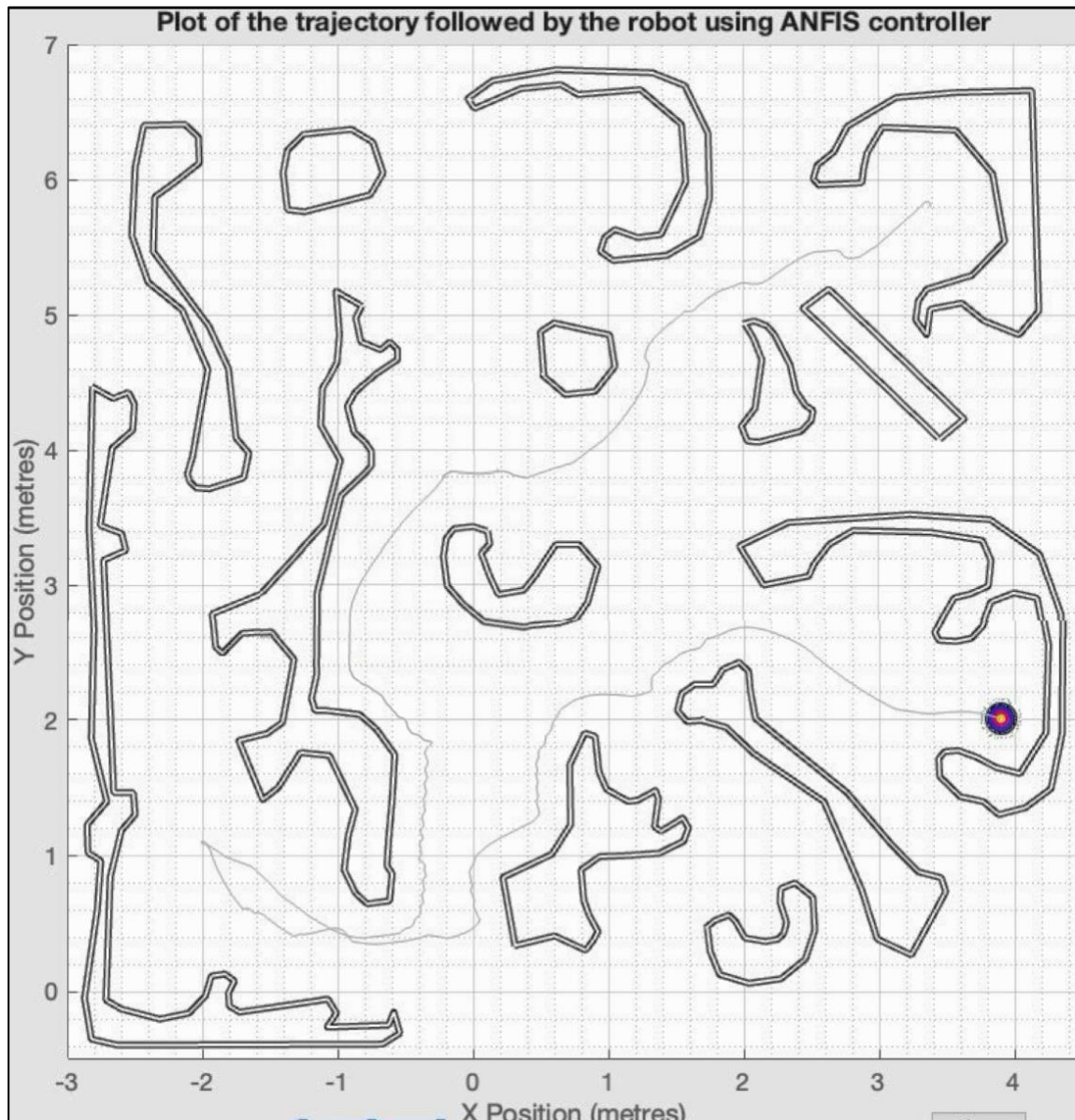


Figure 3.4 - Trajectory of the robot using ANFIS controller

The plots of the input and output signals captured by the scope are shown in *Figure 5.5, Appendix B*. As expected, the graphs were more stable. Although theta exhibited occasional spikes, the signal appeared significantly less noisy and oscillatory compared to the fuzzy controller. The overall signal patterns resemble those of the fuzzy system but differ slightly due to the alternate path taken by the robot. Notably, the ANFIS signals demonstrate greater consistency over time.

The rule surfaces of the ANFIS controller are also more detailed, featuring smooth curves instead of the sharp, flat plateaus seen in the fuzzy controller. The left and right wheels are also not mirror images of each other; they are much more tailored and optimised. These surfaces for each wheel are displayed in *Figure 5.6* and *Figure 5.7* in *Appendix B*.

4. Conclusion

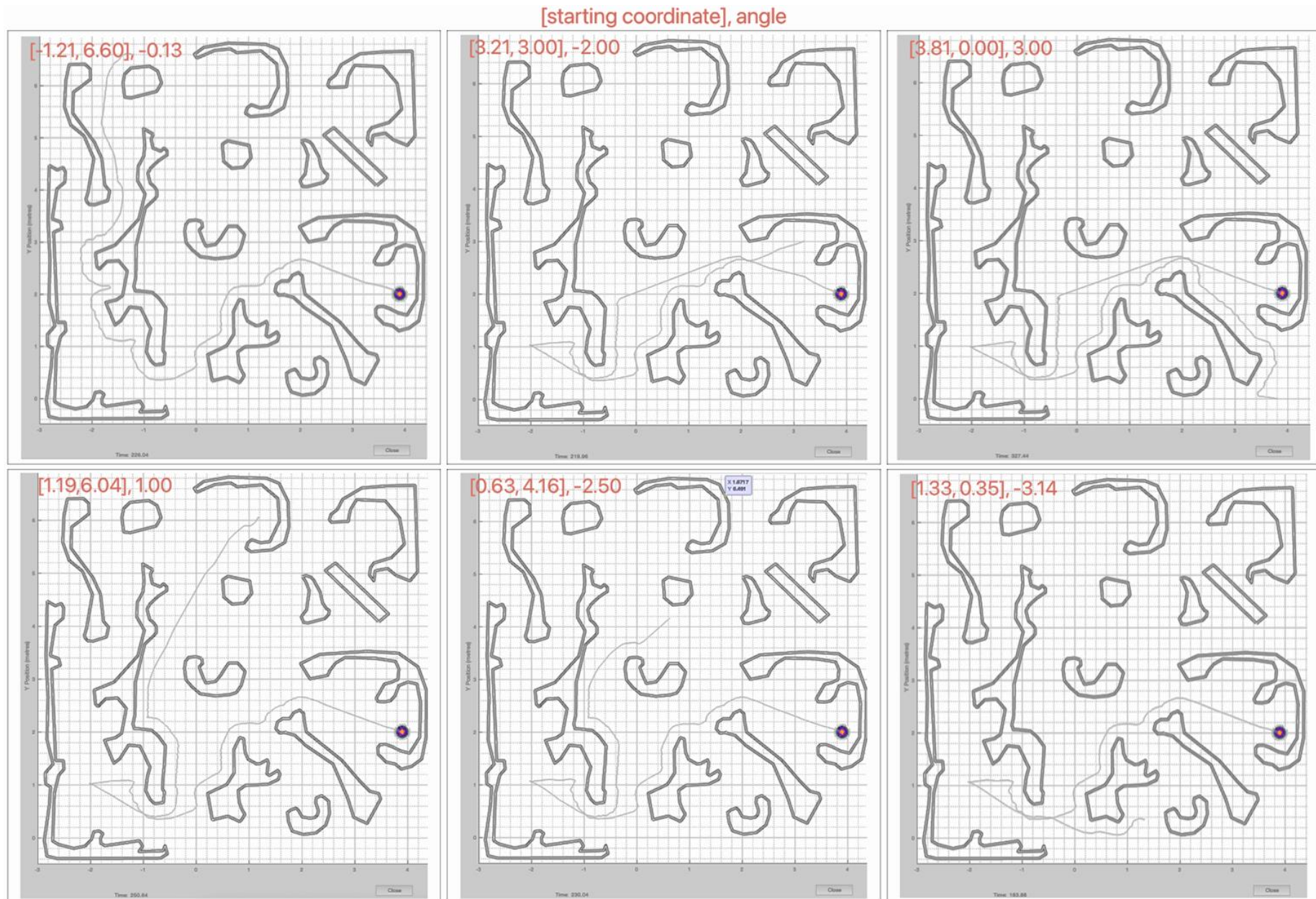
Overall, the project was highly successful, providing the student with a valuable understanding of Adaptive Neural Fuzzy Inference Systems (ANFIS) and how they compare to manually designed fuzzy controllers. It is acknowledged that more advanced and optimal solutions may be designed using the many features that are available in Simulink, however, this would need much more practice, experience and expertise. The initial stages of the project demanded substantial time, experimentation, and iterative tuning to understand how the system behaves and to refine the controllers accordingly.

The strength of neural-fuzzy systems was demonstrated by the trained model's effective performance in the obstacle course. However, it is also evident that ANFIS controllers are much less intuitive and far more complex; retuning or modifying them is not as straightforward as with fuzzy controllers.

Both controllers successfully reached their targets, using different approaches and showed consistency in their ability when the simulation was re-run from different locations and orientations. While the ANFIS controller produced smoother, more stable movements and filtered out noise, it did take a longer route to the target. It is likely that significantly more training—potentially thousands of epochs—would be needed to achieve error convergence, for the system would be much more intelligent and effective.

5. Appendix

A. Task 1: Fuzzy Control Simulation



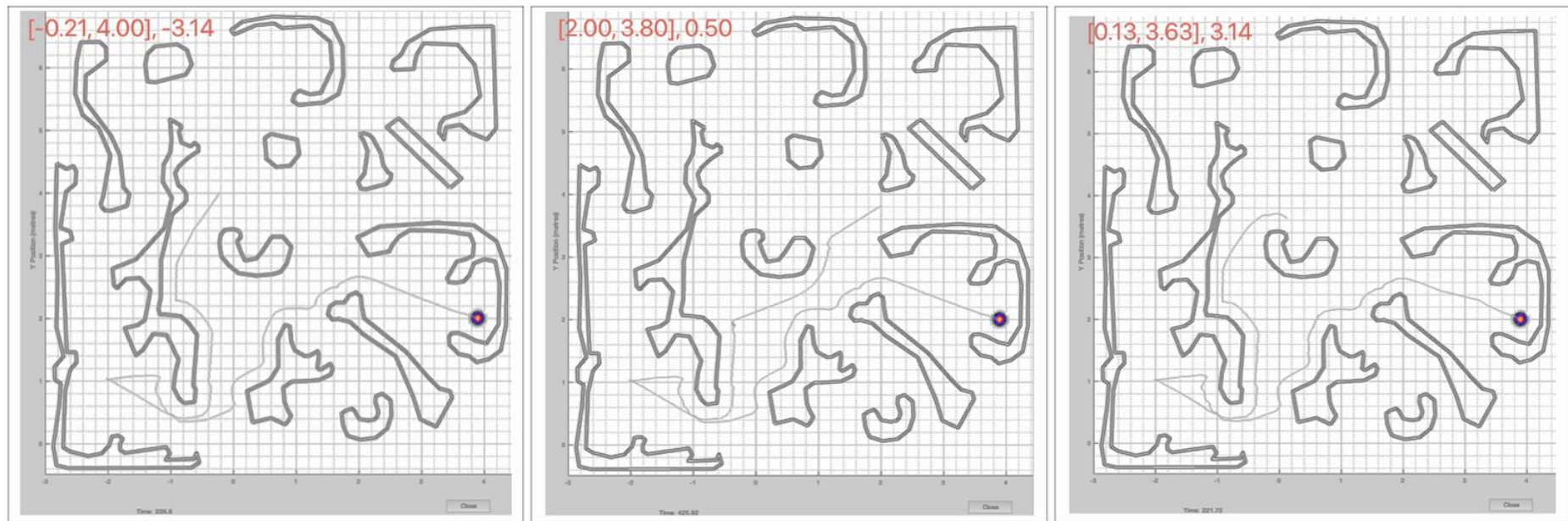


Figure 5.1 - Nine additional runs from different locations at different angles

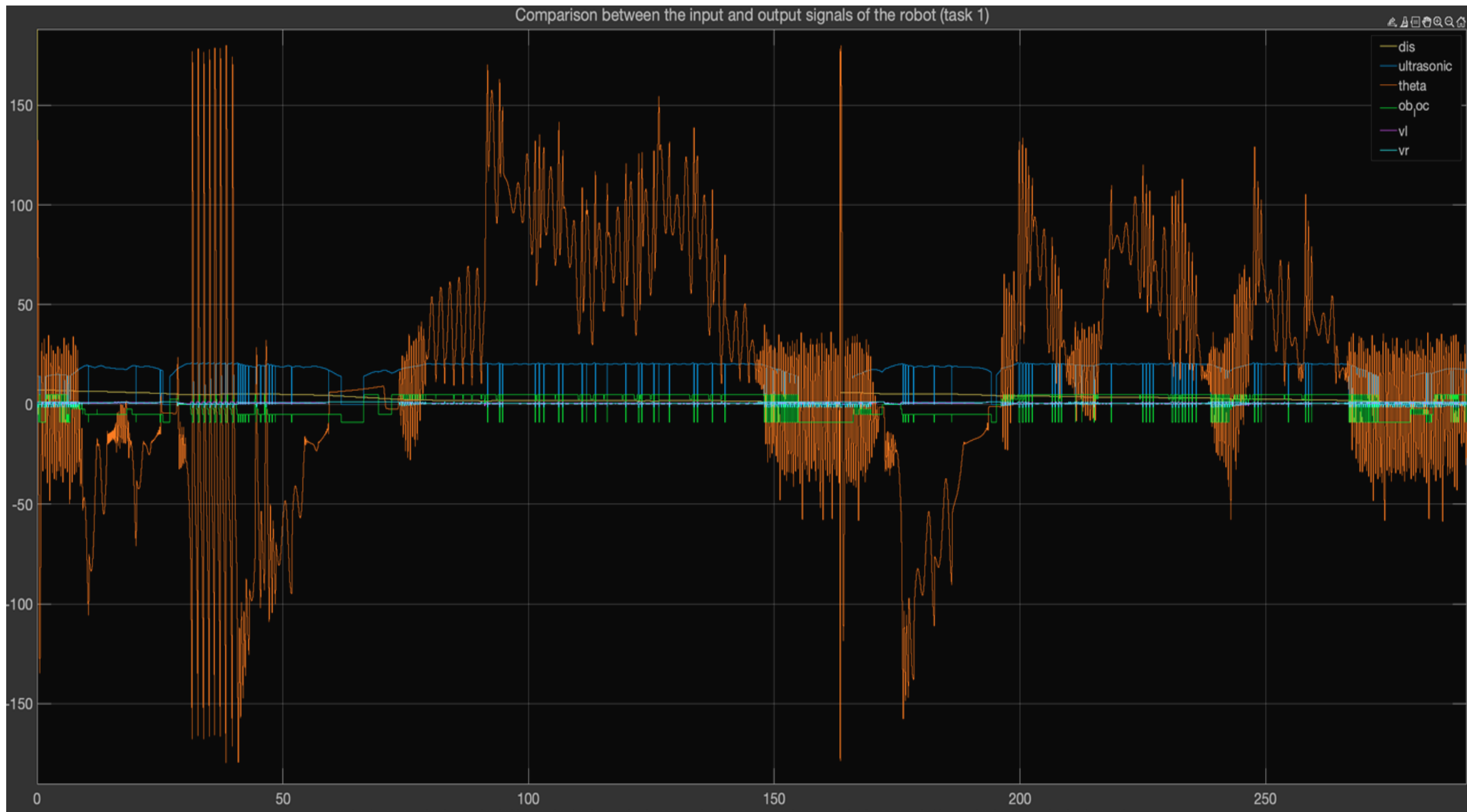


Figure 5.2 - Input and output signals of the robot with a fuzzy controller

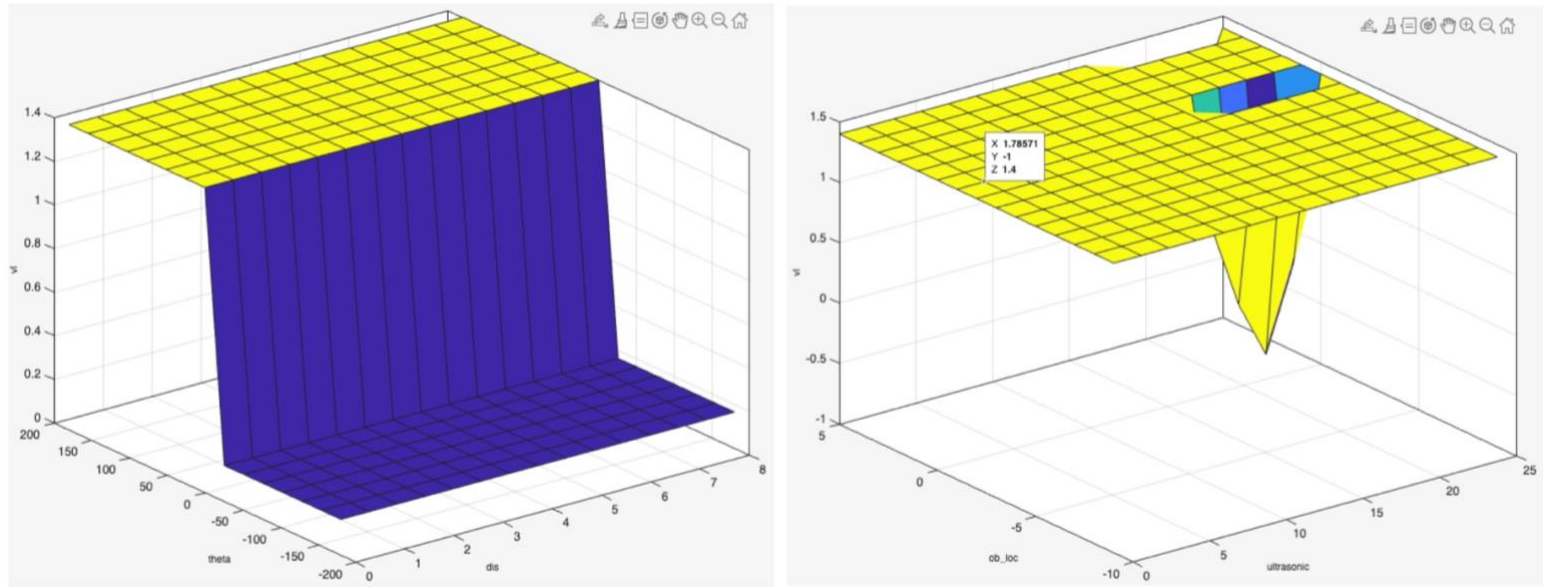


Figure 5.3 – Fuzzy rule surface view of left wheel: i) vl plotted against θ (x-axis) and dis (y-axis), ii) vl plotted against ob_loc (x-axis) and $ultrasonic$ (y-axis) - (right)

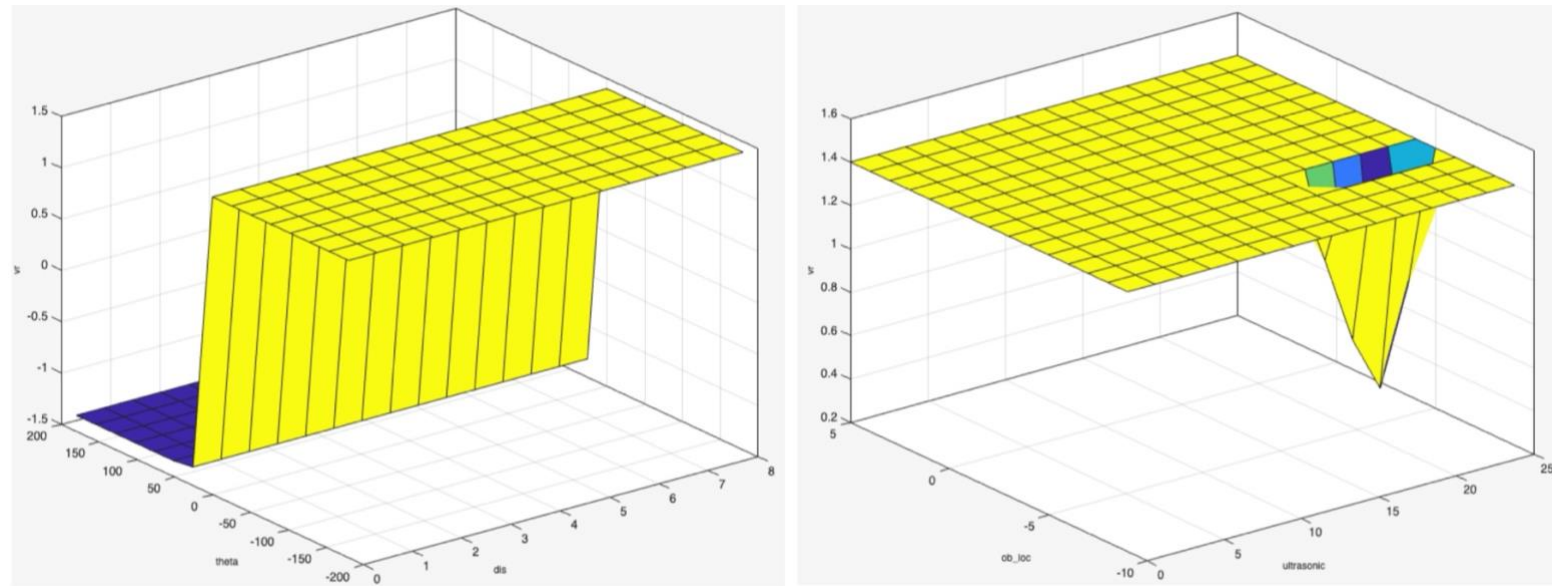


Figure 5.4 – Fuzzy rule surface view of right wheel: i) vr plotted against θ (x-axis) and dis (y-axis), ii) vr plotted against ob_loc (x-axis) and $ultrasonic$ (y-axis) - (right)

B. Task 2: Fuzzy ANFIS Control

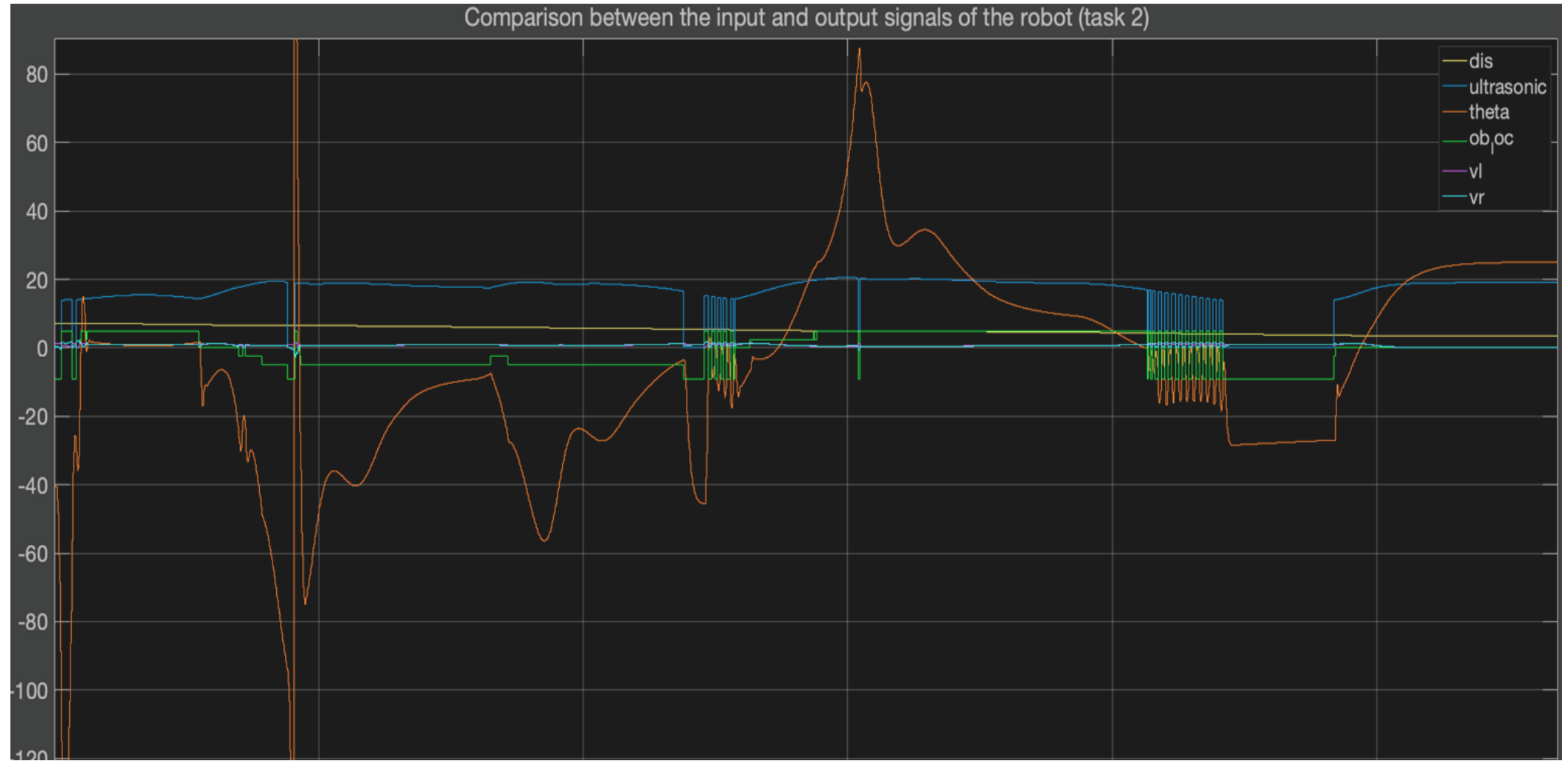


Figure 5.5 - Input and output signals of the robot with ANFIS controller

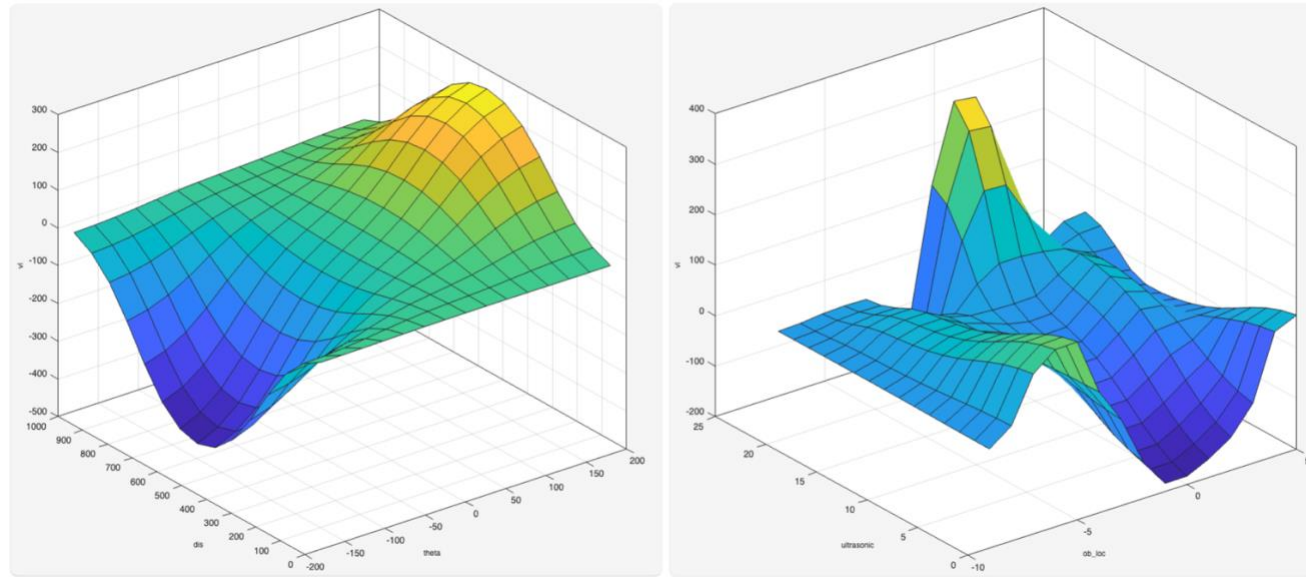


Figure 5.6 - ANFIS rule surface view of left wheel: i) vl plotted against theta (x-axis) and dis (y-axis), ii) vl plotted against ob_loc (x-axis) and ultrasonic (y-axis) - (right)

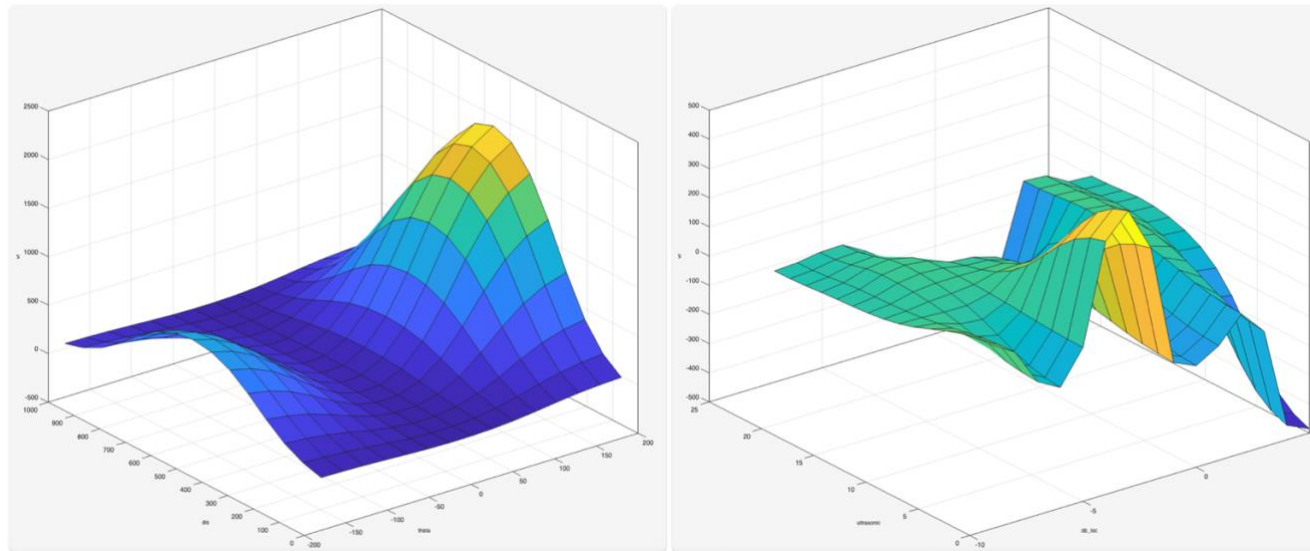


Figure 5.7 - ANFIS rule surface view of right wheel: i) vr plotted against theta (x-axis) and dis (y-axis), ii) vr plotted against ob_loc (x-axis) and ultrasonic (y-axis) - (right)